COMP 2011 Midterm Exam - Spring 2024 - HKUST
Question Book

Date: March 23, 2024 (Saturday)
Time Allowed: 2 hours, 5pm – 7pm
Instructions:

1. This is a closed-book, closed-notes examination.
2. There are **7** questions on **18** pages (including this cover page and 3 blank pages at the end).
3. Write your answers in the answer book provided in black/blue ink. *NO pencil please, otherwise you are not allowed to appeal for any grading disagreements.*
4. All programming codes in your answers must be written in the ANSI C++11 version as taught in the class.
5. For programming questions, you are **NOT** allowed to define additional helper functions or structures, nor global variables, unless otherwise stated. You **cannot** use any library functions not mentioned, nor the `auto` keyword in defining identifiers in the questions.

Student Name	
Student ID	
Venue + Seat Number	

Problem	**Score**
Loops	8
enum and switch	9
Function Parameter Passing	9
Finding Bugs	12
Recursion	10
2D Array and Functions	22
C Strings	30
Total	100

Problem 1 [8 points] Loops

```
bool check_sth(int x)
{
   if (x < 0)
      return false;

   int y = 0, temp = x;

   while (temp)
   {
      y *= 10;
      y += (temp % 10);
      temp /= 10;
   }
   return (y == x);
}
```

Given the above function, what is the returned value for each of the following function calls?

(a) `check_sth(153)`

(b) `check_sth(13531)`

Problem 2 [9 points] enum and switch

```
#include <iostream>
using namespace std;

int main()
{
   enum Grade {A, B, C = 3, D};

   char c;
   cin >> c;

   switch (c - 'a')
   {
      case A:
         cout << "Excellent" << endl;
         break;
      case B:
         cout << "Very Good" << endl;
         break;
      case C:
         cout << "Good" << endl;
         break;
      case D:
         cout << "Satisfactory" << endl;
         break;
      default:
         cout << "Pass" << endl;
   }
   return 0;
}
```

For each input below, what is the corresponding output of the above program?

(a) **Input:** b

(b) **Input:** d

(c) **Input:** 2

Problem 3 [9 points] Function Parameter Passing

```
1  #include <iostream>
2  using namespace std;
3
4  /* This comment is to be replaced by different function definition of change */
5
6  void mystery(int arr[], int start, int end)
7  {
8     int mid = (start + end) / 2;
9
10    if (arr[mid] < arr[start])
11       change(arr[start], arr[mid]);
12
13    if (arr[end] < arr[start])
14       change(arr[start], arr[end]);
15
16    if (arr[end] < arr[mid])
17       change(arr[mid], arr[end]);
18 }
19
20 int main()
21 {
22    const int NUM = 4;
23    int arr[NUM] = {7, 9, 1, 3};
24
25    mystery(arr, 0, NUM-1);
26
27    for (int i = 0; i < NUM; i++)
28       cout << arr[i] << " ";
29    cout << endl;
30    return 0;
31 }
```

When the comment line #4 of the above program is replaced by each of the following function definitions, give the corresponding output.

(a) `void change(int a, int b) { int temp = a; a = b; b = temp; }`

(b) `void change(int a, int& b) { int temp = a; a = b; b = temp; }`

(c) `void change(int& a, int& b) { int temp = a; a = b; b = temp; }`

Problem 4 [12 points] Finding Bugs

In each part below, there is a compilation error. Write down the line number of the code that causes the error, and give a brief explanation of why the program cannot be compiled.

(a)

```
1  #include <iostream>
2  using namespace std;
3
4  void add_one(int& arr[], int size)
5  {
6     for (int i = 0; i < size; ++i)
7        arr[i]++;
8  }
9
10 int main()
11 {
12    int numbers[3] = {1, 2, 3};
13    add_one(numbers, 3);
14    return 0;
15 }
```

(b)

```
1  #include <iostream>
2  using namespace std;
3
4  void set(char arr[])
5  {
6     arr[0] = 'g';
7  }
8
9  int main()
10 {
11    char arr[2][] = { "c ++", "cold" };
12    set(arr[1]);
13    cout << arr[1] << endl;
14    return 0;
15 }
```

(c)

```
#include <iostream>
using namespace std;

void add_one(double& x)
{
   x += 1;
}

int main()
{
  int n = 10;
  add_one(n);
  cout << n << endl;
  return 0;
}
```

(d)

```
#include <iostream>
using namespace std;

int add_one(const int& a)
{
   ++a;
   return (a);
}

int main()
{
   int a = 10;
   add_one(add_one(a));
   return 0;
}
```

Problem 5 [10 points] Recursion

What is the output of the following program?

```
#include <iostream>
using namespace std;

const int N = 4;

void print(const int A[], int n)
{
    if (n < 1)
        return;

    int temp[N] = {};
    for (int i = 0; i < n - 1; i++)
        temp[i] = A[i] + A[i + 1];

    print(temp, n - 1);

    for (int i = 0; i < n; i++)
        cout << A[i] << '\t';

    cout << endl;
}

int main()
{
    int A[N] = {1, 2, 3, 4};
    print(A, N);
    return 0;
}
```

Problem 6 [22 points] 2D Array and Function

Sudoku is a number-placement puzzle. In classic Sudoku, the objective is to fill a 9×9 grid with digits so that each column, each row, and each of the nine 3×3 subgrids contains all the digits from 1 to 9. The puzzle setter provides a partially completed grid which, for a well-posed puzzle, has a single solution.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

A typical Sudoku puzzle

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

The solution to the puzzle

The basic rules of Sudoku:

- Sudoku grid consists of 9×9 spaces.
- You can use only numbers from 1 to 9.
- Each 3×3 subgrid can only contain numbers from 1 to 9.
- Each horizontal row can contain only numbers from 1 to 9.
- Each vertical column can only contain numbers from 1 to 9.
- Each number in a 3×3 subgrid, vertical column or horizontal row can be used only once.
- The game ends when the whole Sudoku grid is correctly filled with numbers.

We can write a program to implement the Sudoku game. Complete the following incomplete program by writing the three functions required in parts (a)-–(c).

```cpp
#include <iostream>
using namespace std;

const int N = 9;
const char BLANKSPACE = ' ';
const char COLBREAK[] = "| ";
const char LINEBREAK[] = "------+-------+------";

int main()
{
    // 0 is for empty cell in Sudoku puzzle
    int data[N * N] = {5, 3, 0, 0, 7, 0, 0, 0, 0,
                       6, 0, 0, 1, 9, 5, 0, 0, 0,
                       0, 9, 8, 0, 0, 0, 0, 6, 0,
                       8, 0, 0, 0, 6, 0, 0, 0, 3,
                       4, 0, 0, 8, 0, 3, 0, 0, 1,
                       7, 0, 0, 0, 2, 0, 0, 0, 6,
                       0, 6, 0, 0, 0, 0, 2, 8, 0,
                       0, 0, 0, 4, 1, 9, 0, 0, 5,
                       0, 0, 0, 0, 8, 0, 0, 7, 9};
    // Sudoku grid
    int grid[N][N];

    cout << "Welcome to the Sudoku game:" << endl;

    loadSudoku(data, grid); // task a)
    printSudoku(grid);      // task b)
    cout << endl;

    int row, col, num;
    while (!isFull(grid))
    { //check whether the grid is full, e.g. no 0
        cout << "input row, col, num" << endl;
        cin >> row >> col >> num;
        if (isValidNum(grid, row, col, num)) // task c)
        {
            grid[row][col] = num;
            cout << "grid[" << row << "][" << col << "] = " << num << endl;
            printSudoku(grid);
        }
        else
            cout << "Invalid number" << endl;
    }

    if (isValidSudoku(grid))
    { // check whether a Sudoku is done
        cout << "Congratulations! You solved the Sudoku!" << endl;
        printSudoku(grid);
    }
    else
        cout << "Oops, something is wrong." << endl;

    return 0;
}
```

Note:

- Although in the given main program, the puzzle is initialized with some hard-coded values, your solution should work with any properly initialized Sudoku puzzles.
- You only need to implement functions in tasks (a) to (c), and assume that the other functions have been implemented.

Below is a sample output of the Sudoku program.

```
Welcome to the Sudoku game:
5 3 0 | 0 7 0 | 0 0 0
6 0 0 | 1 9 5 | 0 0 0
0 9 8 | 0 0 0 | 0 6 0
------+-------+------
8 0 0 | 0 6 0 | 0 0 3
4 0 0 | 8 0 3 | 0 0 1
7 0 0 | 0 2 0 | 0 0 6
------+-------+------
0 6 0 | 0 0 0 | 2 8 0
0 0 0 | 4 1 9 | 0 0 5
0 0 0 | 0 8 0 | 0 7 9

input row, col, num
0 2 4
grid[0][2] = 4
5 3 4 | 0 7 0 | 0 0 0
6 0 0 | 1 9 5 | 0 0 0
0 9 8 | 0 0 0 | 0 6 0
------+-------+------
8 0 0 | 0 6 0 | 0 0 3
4 0 0 | 8 0 3 | 0 0 1
7 0 0 | 0 2 0 | 0 0 6
------+-------+------
0 6 0 | 0 0 0 | 2 8 0
0 0 0 | 4 1 9 | 0 0 5
0 0 0 | 0 8 0 | 0 7 9
input row, col, num
1 1 10
Invalid number
input row, col, num
1 1 -9
Invalid number
input row, col, num
1 1 3
Invalid number
input row, col, num
2 0 8
```

```
Invalid number
input row, col, num
1 2 9
Invalid number
input row, col, num

.......                      // several rounds of user inputs omitted
.......
.......

Congratulations! You solved the Sudoku!
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
------+-------+------
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
------+-------+------
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
```

(a) *(4 points)* Implement the function `loadSudoku`, which initializes a 2D Sudoku `grid[N][N]` from 1D `data[N]`.

(b) *(8 points)* Implement the function `printSudoku`, which prints the 2D Sudoku grid. Refer to the sample output below. Make sure your implementation can produce exactly the same output in the same format.

```
5 3 0 | 0 7 0 | 0 0 0
6 0 0 | 1 9 5 | 0 0 0
0 9 8 | 0 0 0 | 0 6 0
------+-------+------
8 0 0 | 0 6 0 | 0 0 3
4 0 0 | 8 0 3 | 0 0 1
7 0 0 | 0 2 0 | 0 0 6
------+-------+------
0 6 0 | 0 0 0 | 2 8 0
0 0 0 | 4 1 9 | 0 0 5
0 0 0 | 0 8 0 | 0 7 9
```

(c) *(10 points)* Implement the function `isValidNum`, which checks whether the input number, `num`, is valid for the specified spot (row, col) in the current Sudoku grid.
Note:

- You may assume that the inputs, `row` and `col`, are always valid (e.g. locate an empty spot in the Sudoku grid).
- Read carefully the basic rules of the Sudoku game. Check program output to understand how to decide a `num` is valid or not.

Problem 7 [30 points] C Strings

C++ uses 1D character array with the null character end-marker to represent strings which are called C strings. All character arrays in this question are used to store C strings of length less than N. You are required to write **5** functions to deal with C strings with the given function headers. Below is an example program that demonstrates how they work. Your solution should work for any value of N though the program set it to 100.

Note: You are **not** allowed to define or create any additional arrays and helper function(s), nor include additional libraries. In particular, you are NOT allowed to use any C string functions in the cstring library. On the other hand, a function required by a later part may call functions written for earlier parts.

```
#include <iostream>
using namespace std;
const char NULL_CHAR = '\0';

/* THE REQUIRED FUNCTIONS ARE SUPPOSED TO BE WRITTEN HERE */

int main()
{
    const int N = 100;
    char s1[N], s2[N], str_sum[N], pattern[N];
    int start, end, length;

    while (true)
    {
        cout << "Enter a string : ";
        cin >> s1;
        cout << "Enter the substring to search in the form of \"a*b\": ";
        cin >> pattern;

        length = substring(s1, pattern, start, end);
        cout << "substring: " << length << " ; " << start << " , " << end << endl;

        just_letters(s1);
        cout << "s1 after calling just_letters: " << s1 << endl;
        to_lower_case(s1);
        cout << "s1 after calling to_lower_case: " << s1 << endl;

        cout << "Enter another string to add: ";
        cin >> s2;
        just_letters(s2);
        to_lower_case(s2);
        add_strings(s1, s2, str_sum);
        cout << str_sum endl << "\n\n--------------------------------" << endl;
    }

    return 0;
}
```

A sample run of the program is shown below.

```
Enter a string : hongkonguniversity
Enter the substring to search in the form of "a*b": n*k
substring: 3 ; 2 , 4
s1 after calling just_letters: hongkonguniversity
s1 after calling to_lower_case: hongkonguniversity
Enter another string to add: AAA
ipogkonguniversity

--------------------------------
Enter a string : hongkonguniversity
Enter the substring to search in the form of "a*b": n*n
substring: 8 ; 2 , 9
s1 after calling just_letters: hongkonguniversity
s1 after calling to_lower_case: hongkonguniversity
Enter another string to add: aaabbb
ipoimqnguniversity

--------------------------------
Enter a string : 2024Scomp2011LA5##
Enter the substring to search in the form of "a*b": 4*2
substring: 7 ; 3 , 9
s1 after calling just_letters: ScompLA
s1 after calling to_lower_case: scompla
Enter another string to add: ZZZZzzzz
scomplaz

--------------------------------
Enter a string : 2024Scomp2011LA5##
Enter the substring to search in the form of "a*b": S*1
substring: 9 ; 4 , 12
s1 after calling just_letters: ScompLA
s1 after calling to_lower_case: scompla
Enter another string to add: abcd
terqpla

--------------------------------
Enter a string : 2024Scomp2011LA5##
Enter the substring to search in the form of "a*b": S*c
substring: 2 ; 4 , 5
s1 after calling just_letters: ScompLA
s1 after calling to_lower_case: scompla
Enter another string to add: yyyyY
rbnlola

--------------------------------
```

(a) A function with the following header:

```
bool is_letter(char c)
```

`is_letter` takes a character as input, checks if it is a (lower-case or upper-case) letter in the English alphabet, and returns true if it is, otherwise false.

(b) A function with the following header:

```
bool to_lower_case(char s[])
```

`to_lower_case` takes a C string as input that consists of just English letters, and converts any of its upper-case letters to their lower-case letters.

(c) A function with the following header:

```
int substring(const char s[], const char pattern[], int& start, int& end)
```

`substring` takes in two C strings, **s** and **pattern**. The pattern always contains 3 characters of the form **a*b**, where **a** and **b** represent any ASCII characters which are not necessarily English letters, and they may even be the same. Here, * means zero or more ASCII characters. The function looks for the **longest match** of the pattern **a*b** in the C string **s**, and records the first appearance of the character represented by `pattern[0]` in the parameter **start** and the last appearance of the character represented by `pattern[2]` in the parameter **end** (and `pattern[1]` is always '*'). It also returns the length of the matched substring.

(d) A function with the following header:

```
void just_letters(char s[])
```

`just_letters` removes all characters in the input C string **s** that are not letters, and shifts all of its letters forward to the spaces left by the removed characters. After the function is called and when **s** is printed, only the letters in the original C string will be printed and their order should be the same as their order before the function call.

(e) A function with the following header:

```
void add_strings(const char s1[], const char s2[], char str_sum[])
```

`add_strings` adds the corresponding characters of the 2 input C strings **s1** and **s2** to the parameter **str_sum**. The two C strings may have different lengths, and the remaining characters of the longer C string will be copied to **str_sum**.

Assumption: The two input C strings, **s1** and **s2**, contain only letters in lower case.

---------- END OF PAPER ----------

/* Rough work */

/* Rough work */

/* Rough work */